

MULTIPLUS ACADEMY LMS

Analise Completa do Dashboard do Aluno

Estado Actual, Correcoes Pendentes e Plano de Refatoracao

Versao 019 | 16 de Julho de 2026

Autor: Super Z — Orientador de Desenvolvimento

1. Resumo Executivo

Este documento apresenta uma análise abrangente do Dashboard do Aluno do MultiPlus Academy LMS, incluindo o estado actual do código, a verificação das correções solicitadas nos documentos 017 e 018, a integração das mudanças do banco de dados Supabase (migration 004), e um plano detalhado para a refatoração do componente monolítico StudentPortal.tsx. A análise foi conduzida após a aplicação das migrations no Supabase e a revisão completa de todos os ficheiros relacionados com a experiência do aluno na plataforma.

O dashboard do aluno é o módulo mais complexo da aplicação, contendo o componente StudentPortal.tsx com 1925 linhas de código, 4 sub-componentes na pasta portal/, e integração com 7+ serviços Supabase. A análise revelou que embora várias correções críticas dos documentos anteriores tenham sido aplicadas com sucesso (como a remoção de dados mock-multiplus e a adição dos tipos Assignment/AssignmentSubmission), existem ainda 8 correções pendentes e uma refatoração estrutural necessária para garantir a qualidade e manutenibilidade do código a longo prazo.

Em termos de gravidade, os problemas identificados podem ser categorizados em três níveis: críticos (bugs que impedem funcionalidade core como aulas bloqueadas indevidamente), altos (dados fictícios que comprometem a credibilidade do sistema), e médios (problemas de performance e manutenibilidade que afetam a experiência de desenvolvimento). A resolução destes itens deve seguir a ordem de prioridade estabelecida neste relatório.

2. Estado das Correções do Documento 017

O documento 017 especificava 5 correções pendentes pos-auditoria. A análise do código actual revela que 3 das 5 correções foram aplicadas com sucesso, enquanto 2 permanecem pendentes. A tabela seguinte apresenta o estado detalhado de cada item, incluindo a localização exacta no código quando relevante.

Correção	Descrição	Estado	Detalhes
Correção 1	Remover links mock-multiplus	Aplicada	Nenhuma ocorrência de mock-multiplus encontrada no código-fonte. Todas as 4 edições foram realizadas com sucesso.
Correção 2	Remover total_lessons: 3 hardcoded	Aplicada	O fallback em academicService agora busca contagem real de aulas via Supabase. Nenhuma ocorrência de total_lessons: 3 encontrada.
Correção 3	Remover thumbnail Unsplash hardcoded	Aplicada	Nenhuma URL Unsplash encontrada em academicService.ts. O fallback agora usa null.
Correção 4	Adicionar tipos Assignment e AssignmentSubmission	Aplicada	Ambos os tipos foram adicionados ao ficheiro src/types.ts (linhas 129-151). StudentTasksTab.tsx importa Assignment e AssignmentSubmission correctamente.
Correção 5	Remover antonio@advogados.ao	Aplicada	Nenhuma ocorrência de antonio@advogados.ao encontrada no código-fonte. A referência foi removida do InstructorEvaluationsTab.

Conclusão: Todas as 5 correções do documento 017 foram aplicadas com sucesso. O código-fonte está limpo de dados fictícios críticos como URLs mock-multiplus, emails hardcoded, e contagens

ficticias de aulas. Este e um progresso significativo que valida a eficacia do processo de auditoria.

3. Correcoes Pendentes do Documento 018 — Seccao A

O documento 018 define 8 correcoes na Seccao A (bugs e dados ficticios remanescentes) e uma refatoracao estrutural na Seccao B. A analise revelou que nenhuma das correcoes da Seccao A foi aplicada ate ao momento. Abaixo, o estado detalhado de cada item com evidencia extraida directamente do codigo-fonte.

3.1 A1 — BUG CRITICO: Aulas sem `scheduled_at` bloqueadas

Este e o bug mais grave identificado. Na linha 1509 do `StudentPortal.tsx`, a logica de bloqueio das aulas esta invertida para aulas sem data de agendamento. Quando `scheduled_at` e `null`, a aula deveria estar desbloqueada (disponivel imediatamente), mas a logica actual marca como bloqueada (`true`), impedindo o aluno de ver QUALQUER aula que nao tenha data agendada.

Codigo actual (INCORRETO):

```
const isLocked = syll.scheduled_at ? new Date(syll.scheduled_at) > new Date() : true;
```

Codigo corrigido:

```
const isLocked = syll.scheduled_at ? new Date(syll.scheduled_at) > new Date() : false;
```

Impacto: Este bug impede que os alunos acessem a aulas publicadas sem data agendada, o que e o cenario mais comum num LMS onde o conteudo esta disponivel por padrao. A correcao e trivial (mudar `true` para `false`), mas o impacto e critico na experiencia do utilizador.

3.2 A2 — Dados ficticios no PDF Export

A funcao `handleExportPDF` no `StudentPortal.tsx` (linhas 414-453) ainda contem dois blocos de dados ficticios como fallback. O primeiro (linhas 414-421) cria um `enrollment` ficticio com `courseId` "eng-legal-angola" e `progressPercent`: 66. O segundo (linhas 424-437) cria um certificado ficticio para utilizadores cujo email contenha "antonio". Adicionalmente, na linha 453, o fallback de progresso esta hardcoded como 66 em vez de 0.

Estes dados ficticios no PDF exportado comprometem seriamente a credibilidade do sistema, pois o aluno pode exportar um documento academico oficial que contenha informacoes falsas. A correcao deve substituir os fallbacks ficticios por arrays vazios e o `progressPercent` de fallback deve ser 0 em vez de 66.

3.3 A3 — Fallbacks ficticios no `StudentProgressTab`

O ficheiro `StudentProgressTab.tsx` ainda contem valores hardcoded como fallback nas linhas 44-49. A `streak` (ofensiva) tem fallback para 5, as horas de estudo tem fallback para 24, e o total de aulas tem fallback para 3. Estes valores ficticios apresentam ao aluno uma visao distorcida do seu progresso real, mostrando dados que nao correspondem a realidade do banco de dados.

Codigos identificados no codigo-fonte actual:

```
Linha 44: const streak = currentUser?.streak || 5;  
Linha 45: const hours = currentUser?.totalHoursLearned || 24;  
Linha 48: const totalLessons = metrics?.total_lessons ?? 3;
```

3.4 A4/A5 — Telefone e Streak ficticios no perfil

No `StudentPortal.tsx`, o formulario de perfil ainda contem dados ficticios como fallback. Na linha 245, o telefone tem fallback `"+244 923 000 000"` em vez de string vazia. Na linha 251, o `streakCount` tem fallback 5 em vez de 0. Estes valores sao apresentados ao aluno como se fossem dados reais, o que pode causar confusao e falta de confianca na plataforma.

3.5 A6 — Google Calendar Sync falso

A verificacao confirmou que a funcao `handleSyncGoogleCalendar` e o estado `isGoogleSynced` foram removidos do codigo actual. Nenhuma ocorrencia destes termos foi encontrada, indicando que esta correccao ja foi aplicada numa iteracao anterior.

3.6 A7 — Notificacoes vazias (nunca carregadas do Supabase)

O estado de notificacoes no `StudentPortal.tsx` (linha 149) e inicializado como array vazio e nunca e preenchido com dados reais do Supabase. O dropdown de notificacoes mostra sempre vazio, apesar de a tabela `"notifications"` existir no banco de dados e ser alimentada por triggers (como o `trg_notify_new_assignment` na migration 004). Esta funcionalidade e importante para a comunicacao entre professor e aluno.

3.7 A8 — Imagens Unsplash como fallback de avatar

Foram identificadas 13 ocorrencias de URLs Unsplash como fallback em multiplos ficheiros. As localizacoes mais criticas estao nos servicos Supabase e no `AuthProvider`:

Ficheiro	Linha	Tipo
<code>AuthProvider.tsx</code>	91	<code>avatarUrl Unsplash fallback</code>
<code>enrollmentService.ts</code>	144	<code>avatarUrl Unsplash fallback</code>
<code>enrollmentService.ts</code>	231	<code>avatarUrl Unsplash fallback</code>
<code>courseService.ts</code>	97	<code>thumbnail Unsplash fallback</code>
<code>CourseEditorModal.tsx</code>	1094	<code>avatarUrl Unsplash fallback</code>
<code>StudentSelector.tsx</code>	161	<code>avatarUrl Unsplash fallback</code>
<code>InstructorCoursesTab.tsx</code>	278	<code>avatarUrl Unsplash fallback</code>
<code>InstructorStudentsTab.tsx</code>	277	<code>avatarUrl Unsplash fallback</code>
<code>InstructorStudentsTab.tsx</code>	418	<code>avatarUrl Unsplash fallback</code>
<code>InstructorPortal.tsx</code>	214	<code>avatarUrl Unsplash fallback</code>
<code>data.ts</code>	157	<code>thumbnail Unsplash (curso)</code>
<code>data.ts</code>	182	<code>thumbnail Unsplash (curso)</code>
<code>data.ts</code>	202	<code>thumbnail Unsplash (curso)</code>

Nota: O ficheiro `data.ts` pode ser considerado uma excepcao aceitavel, pois serve como dados iniciais de seed para cursos demonstrativos. Os restantes ficheiros devem ser corrigidos substituindo URLs Unsplash por null, garantindo que a interface mostre iniciais do utilizador ou placeholders genericos em vez de fotos aleatorias de estranhos.

3.8 Resumo da Seccao A

ID	Correcao	Estado	Prioridade	Ficheiro
A1	Aulas sem scheduled_at bloqueadas	PENDENTE	CRITICA	StudentPortal.tsx:1509
A2	Dados ficticios no PDF Export	PENDENTE	ALTA	StudentPortal.tsx:414-453
A3	Fallbacks ficticios StudentProgressTab	PENDENTE	ALTA	StudentProgressTab.tsx:44-49
A4	Telefone ficticio no perfil	PENDENTE	MEDIA	StudentPortal.tsx:245
A5	Streak ficticio no perfil	PENDENTE	MEDIA	StudentPortal.tsx:251
A6	Google Calendar Sync falso	APLICADA	—	N/A
A7	Notificacoes nunca carregadas	PENDENTE	ALTA	StudentPortal.tsx:149
A8	Unsplash fallback de avatar (13 locais)	PENDENTE	MEDIA	Multiplos ficheiros

4. Refatoracao Estrutural — Documento 018, Seccao B

A Seccao B do documento 018 propoe uma refatoracao do componente monolitico StudentPortal.tsx (1925 linhas) em sub-componentes e hooks customizados. A analise confirma que nenhuma das refatoracoes propostas foi implementada ate ao momento. O componente StudentPortal.tsx mantem toda a logica inline, incluindo busca de dados, gerenciamento de estado do video, apontamentos, navegacao, exportacao PDF, e renderizacao de todas as abas.

A refatoracao e essencial por tres razoes fundamentais. Primeiro, a performance: qualquer mudanca de estado (como o relógio que actualiza a cada segundo) causa re-renderizacao de todo o componente, incluindo areas nao relacionadas como a lista de aulas ou o formulario de perfil. Segundo, a manutenibilidade: com 1925 linhas, o componente e extremamente dificil de navegar, debugar e testar. Terceiro, a reutilizacao: logica como o player de video e os apontamentos poderia ser reutilizada noutros contextos se extraida para hooks customizados.

4.1 Hooks Customizados Propostos

Hook	Responsabilidade	Estado
useStudentData.ts	Busca de dados Supabase (enrollments, certificates, lessons, notifications, unread messages)	NAO CRIADO
useVideoPlayer.ts	Logica do player de video (progresso, velocidade, marca de agua anti-gravacao)	NAO CRIADO
useLessonNotes.ts	Apontamentos do aluno (carregar, salvar, sincronizar com Supabase)	NAO CRIADO

4.2 Sub-Componentes Propostos

Componente	Responsabilidade	Estado
StudentSidebar.tsx	Sidebar de navegacao com perfil, menu de abas e botao de logout	NAO CRIADO

Componente	Responsabilidade	Estado
StudentTopbar.tsx	Header com busca global, streak, notificacoes e dropdown de perfil	NAO CRIADO
StudentDashboardView.tsx	Aba dashboard com boas-vindas, proxima aula e estatisticas	NAO CRIADO

Apos a refatoracao, o StudentPortal.tsx principal deveria ter menos de 300 linhas, actuando apenas como orquestrador dos sub-componentes e hooks. O documento 018 fornece o codigo completo para os tres hooks customizados (B1-B3) e a especificacao de interfaces para os sub-componentes (B4-B6).

5. Integracao das Mudancas do Banco de Dados

A migration 004 (supabase/migrations/004_video_notes_assignments.sql) foi aplicada com sucesso ao banco de dados Supabase. Esta migration introduziu suporte a tres novas funcionalidades principais que estao directamente integradas com o dashboard do aluno: apontamentos de aula com timestamps de video, sistema de tarefas academicas com submissoes, e registo de progresso de visualizacao de video. Alem disso, foram adicionadas colunas nas tabelas existentes e configuradas politicas de seguranca RLS.

5.1 Tabelas Criadas

Tabela	Descricao	Colunas Principais
lesson_notes	Apontamentos do aluno vinculados a timestamps do video	student_id, lesson_id, course_id, content, video_timestamp
assignments	Tarefas academicas criadas por professores	course_id, lesson_id, teacher_id, titulo, descricao, due_date, status
assignment_submissions	Submissoes de tarefas pelos alunos	assignment_id, student_id, submission_url, submission_text, feedback, grade

5.2 Alteracoes em Tabelas Existentes

Tabela	Coluna	Tipo	Proposito
lesson_progress	video_progress_seconds	INTEGER DEFAULT 0	Regista o segundo actual do video para retomar a reproducao
lessons	meeting_url	TEXT	URL da videoconferencia (Google Meet, Zoom, etc.)

5.3 Seguranca (RLS) e Storage

A migration configurou Row Level Security em todas as novas tabelas com politicas especificas: alunos so podem ver/criar as suas proprias notas e submissoes; professores e admins podem ver dados de alunos dos seus cursos; tarefas publicadas so sao visiveis para alunos matriculados no curso correspondente. O bucket "media" foi criado no Supabase Storage com politicas de upload autenticado e leitura publica, permitindo o upload de ficheiros de submissao de tarefas. A view

vw_student_progress foi recriada para calcular correctamente o progresso com base nas novas tabelas.

5.4 Trigger de Notificacao

A migration criou um trigger (trg_notify_new_assignment) que insere automaticamente registos na tabela notifications quando uma nova tarefa e criada. Este trigger notifica todos os alunos matriculados no curso correspondente. No entanto, conforme identificado na correccao A7, o frontend ainda nao le notificacoes desta tabela, tornando o trigger funcionalmente invisivel para o utilizador final.

5.5 Automacao via GitHub Actions

O workflow de GitHub Actions para deployment automatico de migrations foi preparado, mas requer configuracao manual dos secrets (SUPABASE_ACCESS_TOKEN, SUPABASE_PROJECT_ID, SUPABASE_DB_PASSWORD) no repositorio. Apos esta configuracao, novas migrations na pasta supabase/migrations/ serao deployadas automaticamente ao fazer push para a branch main.

6. Arquitectura Actual do Dashboard do Aluno

O dashboard do aluno e composto pelo componente principal StudentPortal.tsx (1925 linhas) e quatro sub-componentes na pasta portal/. A comunicacao com o Supabase e feita atraves do academicService, que centraliza todas as operacoes de leitura e escrita. Abaixo, a estrutura completa dos ficheiros relacionados.

Ficheiro	Linhas	Descricao	Complexidad e
StudentPortal.tsx	1925	Componente principal (monolitico)	CRITICO
StudentProgressTab.tsx	235	Aba de progresso e metricas	MODERADO
StudentTasksTab.tsx	395	Aba de tarefas e submissoes	MODERADO
StudentMaterialsTab.tsx	237	Aba de materiais de estudo	MODERADO
StudentCertificatesTab.tsx	213	Aba de certificados	MODERADO
QuizArea.tsx	279	Componente de quiz de avaliacao	MODERADO
academicService.ts	597	Servico de dados academicos	ALTO
types.ts	152	Definicoes de tipos TypeScript	MODERADO

6.1 Fluxo de Dados

O fluxo de dados do dashboard segue o padrao: StudentPortal chama academicService que faz queries ao Supabase via client.ts. Os dados sao armazenados em estados React locais (useState) e passados como props para os sub-componentes. Subscricoes em tempo real (Realtime) existem para mensagens nao lidas, mas nao para notificacoes ou progresso de aulas. O sistema de apontamentos e o player de video usam efeitos colaterais (useEffect) para carregar e salvar dados assincronamente, com auto-save a cada 15 segundos para o progresso do video.

6.2 Servicos Integrados

O `academicService` expoe 20+ metodos organizados em 6 categorias: cursos CRUD (criar, ler, actualizar, apagar), sistema de modulos, aulas e materiais, matriculas e progresso, completacoes de aulas, e certificacao. Os metodos mais recentes (adicionados na migration 004) incluem `saveVideoProgress`, `getVideoProgress`, `getLessonNotes`, `saveLessonNote`, `getStudentMaterials`, `getStudentAssignments` e `submitAssignment`. Todos os metodos utilizam o cliente Supabase com tratamento de erros silencioso (`console.error` + retorno de array vazio).

7. Plano de Execucao Recomendado

Com base na analise realizada, recomenda-se a seguinte ordem de execucao, priorizando as correccoes criticas que afectam directamente a experiencia do aluno, seguidas das correccoes de dados ficticios e, por fim, a refatoracao estrutural que melhorara a manutenibilidade a longo prazo.

Fase	Tarefa	Esforco	Priorida de	Justificacao
1	A1: Corrigir <code>isLocked</code> (true para false)	1 linha	CRITICA	Bug impede acesso a aulas
2	A2: Remover dados ficticios do PDF	3 linhas	ALTA	Credibilidade do sistema
3	A3: Corrigir fallbacks <code>StudentProgressTab</code>	4 linhas	ALTA	Visao distorcida do progresso
4	A7: Carregar notificacoes do Supabase	~20 linhas	ALTA	Funcionalidade inexistente
5	A4+A5: Corrigir telefone e streak	2 linhas	MEDIA	Dados falsos no perfil
6	A8: Remover Unsplash fallbacks (10+ locais)	10+ edicoes	MEDIA	Avatares de estranhos
7	B1: Criar <code>useStudentData.ts</code>	~100 linhas	MEDIA	Extrair logica de dados
8	B2: Criar <code>useVideoPlayer.ts</code>	~70 linhas	MEDIA	Extrair logica de video
9	B3: Criar <code>useLessonNotes.ts</code>	~60 linhas	MEDIA	Extrair logica de notas
10	B4-B6: Criar sub-componentes	~300 linhas	MEDIA	Refatoracao UI
11	Refatorar <code>StudentPortal.tsx</code>	Reescrever	MEDIA	Objetivo: <300 linhas
12	<code>npm run build</code> e validacao	Teste	ALTA	Confirmar sem erros

As fases 1-6 (correccoes) podem ser executadas em paralelo por um agente de IA, uma vez que cada correcao e independente e localizada. As fases 7-11 (refatoracao) devem ser executadas sequencialmente, pois cada fase depende da anterior. A fase 12 (validacao) deve ser executada apos cada conjunto de mudancas para garantir que nenhuma regressao foi introduzida.

8. Checklist de Verificacao Final

Apos a implementacao de todas as correccoes e refatoracoes, o seguinte checklist deve ser executado para validar que o dashboard do aluno esta funcional e livre de dados ficticios. Cada

item inclui o comando ou metodo de verificacao recomendado.

#	Verificacao	Metodo
1	Nenhuma aula bloqueada indevidamente sem scheduled_at	Inspecionar isLocked no StudentPortal.tsx
2	Nenhum dado ficticio no PDF export	Procurar "eng-legal-angola", "antonio", "progressPercent: 66"
3	Nenhum fallback ficticio em StudentProgressTab	Verificar streak 0, hours 0, totalLessons ?? 0
4	Telefone e streak sem valores ficticios	Procurar "+244 923" e "streakCount 5"
5	Nenhuma URL Unsplash em services e auth	Procurar "images.unsplash.com" em src/services/ e src/components/auth/
6	Notificacoes carregadas do Supabase	Dropdown mostra notificacoes reais
7	Hooks customizados criados	Verificar existencia de useStudentData.ts, useVideoPlayer.ts, useLessonNotes.ts
8	Sub-componentes criados	Verificar StudentSidebar.tsx, StudentTopbar.tsx, StudentDashboardView.tsx
9	StudentPortal refatorado	Arquivo principal tem menos de 300 linhas
10	Build compila sem erros	Executar npm run build

9. Conclusao

O dashboard do aluno do MultiPlus Academy LMS encontra-se num estado funcional mas com dividas tecnicas significativas. As correccoes do documento 017 foram integralmente aplicadas, demonstrando a eficacia do processo de auditoria iterativa. No entanto, as 7 correccoes pendentes do documento 018 (Seccao A) e a refatoracao estrutural (Seccao B) permanecem como trabalho por realizar.

O bug critico A1 (aulas bloqueadas indevidamente) deve ser corrigido imediatamente, pois afecta directamente a funcionalidade core do LMS. As correccoes de dados ficticios (A2-A5, A8) devem ser implementadas em seguida, pois comprometem a credibilidade do sistema. A funcionalidade de notificacoes (A7) e a refatoracao estrutural (B1-B6) sao melhorias importantes que devem ser agendadas para a proxima sprint.

A integracao da migration 004 no Supabase foi concluida com sucesso, incluindo todas as tabelas, RLS, storage e triggers. O frontend ja consome os novos dados atraves do academicService expandido. A automacao via GitHub Actions esta preparada e aguarda apenas a configuracao manual dos secrets. Com a implementacao das correccoes pendentes e da refatoracao, o dashboard do aluno estara preparado para producao com uma base de codigo limpa, testavel e manutenivel.